

JPA - PART6 (Association Types in JPA)

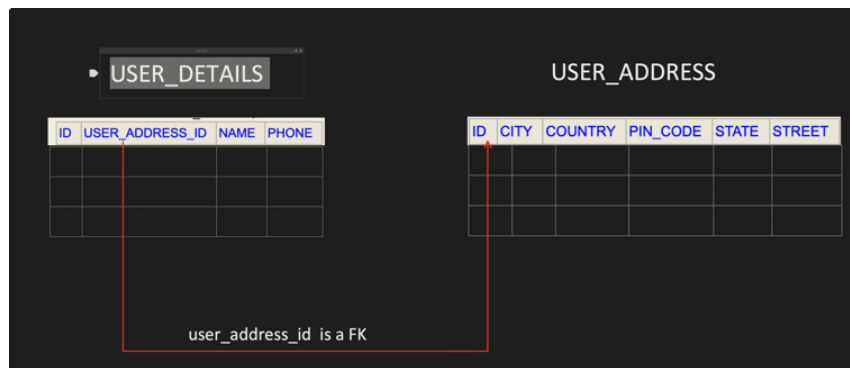
@OneToOne Unidirectional

- One Entity(A) references only one instance of another Entity(B).
- But reference exist only in one Direction i.e. from Parent(A) to Child(B).

```
1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String name;
10    private String phone;
11
12    @OneToOne(cascade = CascadeType.ALL)
13    private UserAddress userAddress;
14
15    // Constructors
16    public UserDetails() {
17    }
18
19    // getters and setters
20 }
21
```

```
1 @Entity
2 @Table(name = "user_address")
3 public class UserAddress {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String street;
10    private String city;
11    private String state;
12    private String country;
13    private String pinCode;
14
15    // Constructors
16    public UserAddress() {
17    }
18 }
19
```

Concept && Coding



- By default hibernate choose:
 - the Foreign Key (FK) name as : <field_name_id>
that's why for "userAddress" it created the FK name as : user_address_id
 - Chooses the Primary Key (PK) of other table.

But If we need more control over it, we can use **@JoinColumn** annotation.

```

1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String name;
10    private String phone;
11
12    @OneToOne(cascade = CascadeType.ALL)
13    @JoinColumn(name = "address_id", referencedColumnName = "id")
14    private UserAddress userAddress;
15
16    // Constructors
17    public UserDetails() {
18    }
19
20    // getters and setters
21 }
22

```

```

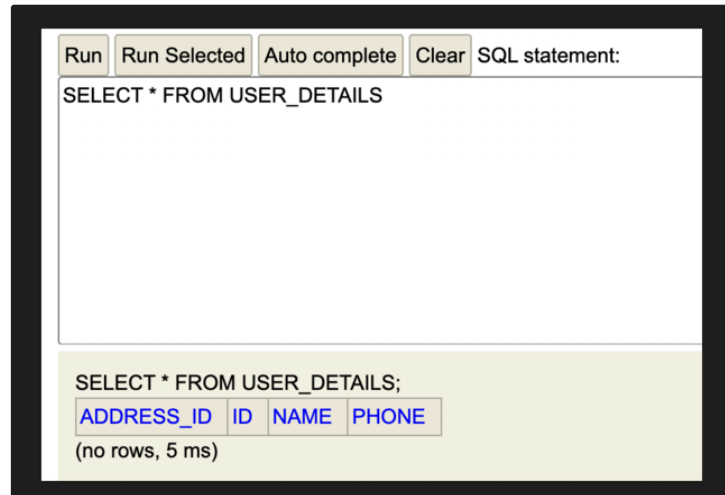
1 @Entity
2 @Table(name = "user_address")
3 public class UserAddress {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String street;
10    private String city;
11    private String state;
12    private String country;
13    private String pinCode;
14
15    // Constructors
16    public UserAddress() {
17    }
18

```

```

19 // getters and setters
20 }
21

```



What about Composite Key, how to reference it?

- We need to use **@JoinColumn** and need to map all columns.

```

1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String name;
10    private String phone;
11
12    @OneToOne(cascade = CascadeType.ALL)
13    @JoinColumns({
14        @JoinColumn(name = "address_street", referencedColumnName =
15"street"),
16        @JoinColumn(name = "address_pin_code", referencedColumnName =
17"pinCode")
18    })
19    private UserAddress userAddress;
20
21    // Constructors
22    public UserDetails() {
23    }
24
25    // getters and setters
26 }

```

```

1 @Entity
2 @Table(name = "user_address")
3 public class UserAddress {
4
5     @EmbeddedId
6     private UserAddressCK id;
7
8     private String city;
9     private String state;
10    private String country;
11
12    // getters and setters

```

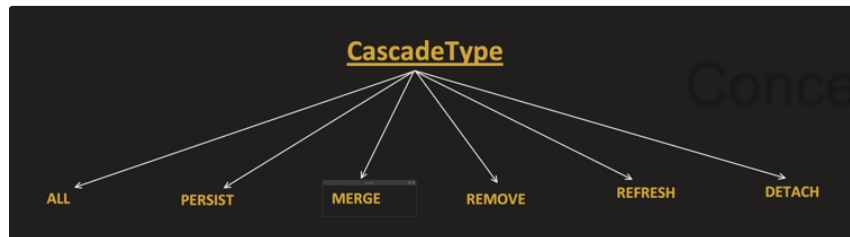
```
13 }
14
```

```
1 @Embeddable
2 public class UserAddressCK {
3
4     private String street;
5     private String pinCode;
6
7     // getters and setters
8 }
9
```

The screenshot shows a database management interface with two SQL query windows. The left window contains the query `SELECT * FROM USER_DETAILS;` and the right window contains `SELECT * FROM USER_ADDRESS;`. Below the queries, there are result tables for both. The `USER_DETAILS` table has columns: ID, ADDRESS_PIN_CODE, ADDRESS_STREET, NAME, PHONE. The `USER_ADDRESS` table has columns: CITY, COUNTRY, PIN_CODE, STATE, STREET. Below these, a larger table shows the results of `SELECT * FROM INFORMATION_SCHEMA.INDEX_COLUMNS;` with columns: INDEX_CATALOG, INDEX_SCHEMA, INDEX_NAME, TABLE_CATALOG, TABLE_SCHEMA, TABLE_NAME, COLUMN_NAME, ORDINAL_POSITION, ORDERING_SPECIFICATION. The table lists indexes for `USER_DETAILS` and `USER_ADDRESS`.

INDEX_CATALOG	INDEX_SCHEMA	INDEX_NAME	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	ORDERING_SPECIFICATION
USERDB	PUBLIC	PRIMARY_KEY_3	USERDB	PUBLIC	USER_DETAILS	ID	1	ASC
USERDB	PUBLIC	CONSTRAINT_INDEX_3	USERDB	PUBLIC	USER_DETAILS	ADDRESS_PIN_CODE	1	ASC
USERDB	PUBLIC	CONSTRAINT_INDEX_3	USERDB	PUBLIC	USER_DETAILS	ADDRESS_STREET	2	ASC
USERDB	PUBLIC	PRIMARY_KEY_9	USERDB	PUBLIC	USER_ADDRESS	PIN_CODE	1	ASC
USERDB	PUBLIC	PRIMARY_KEY_9	USERDB	PUBLIC	USER_ADDRESS	STREET	2	ASC

Now before we proceed with further Mappings, there is one more important thing i.e.



- Without CascadeType, any operation on Parent do not affect Child entity.
- Managing Child entities explicitly can be error-prone.

CascadeType.PERSIST

- Persisting/Inserting the User entity automatically persists its associated UserAddress entity data.

```
1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String name;
10    private String phone;
11
12    @OneToOne(cascade = CascadeType.PERSIST)
13    @JoinColumn(name = "address_id", referencedColumnName = "id")
14    private UserAddress userAddress;
15
16    // Constructors

```

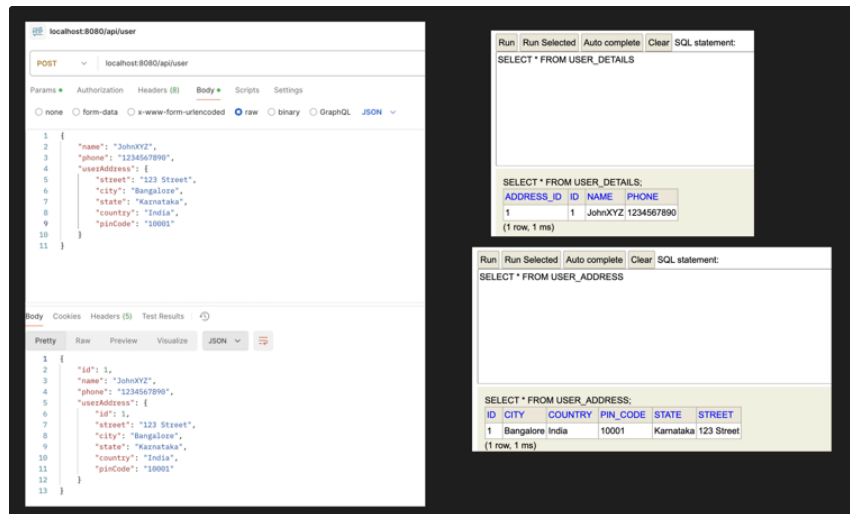
```
17     public UserDetails() {
18     }
19
20     // getters and setters
21 }
22
```

```
1 @Entity
2 @Table(name = "user_address")
3 public class UserAddress {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String street;
10    private String city;
11    private String state;
12    private String country;
13    private String pinCode;
14
15    // getters and setters
16 }
17
```

```
1 @RestController
2 @RequestMapping(value = "/api/")
3 public class UserController {
4
5     @Autowired
6     private UserDetailsService userDetailsService;
7
8     @PostMapping(path = "/user")
9     public UserDetails insertUser(@RequestBody UserDetails
10    userDetails) {
11         return userDetailsService.saveUser(userDetails);
12     }
13 }
```

```
1 @Service
2 public class UserDetailsService {
3
4     @Autowired
5     private UserDetailsRepository userDetailsRepository;
6
7     public UserDetails saveUser(UserDetails user) {
8         return userDetailsRepository.save(user);
9     }
10 }
11
```

```
1 @Repository
2 public interface UserDetailsRepository extends
3 JpaRepository<UserDetails, Long> {
4 }
```



CascadeType.MERGE

- Updating the User entity automatically updates its associated UserAddress entity data.

Lets, first see, what happen if we use only **PERSIT** Cascade type

```

1 @RestController
2 @RequestMapping(value = "/api/")
3 public class UserController {
4
5     @Autowired
6     private UserDetailsService userDetailsService;
7
8     @PostMapping(path = "/user")
9     public UserDetails insertUser(@RequestBody UserDetails
10     userDetails) {
11         return userDetailsService.saveUser(userDetails);
12     }
13
14     @PutMapping(path = "/user/{id}")
15     public UserDetails updateUser(@PathVariable Long id,
16     @RequestBody UserDetails
17     userDetails) {
18         return userDetailsService.updateUser(id, userDetails);
19     }
20 }

```

Concept & Coding

```

1 @Service
2 public class UserDetailsService {
3
4     @Autowired
5     UserDetailsRepository userDetailsRepository;
6
7     public UserDetails saveUser(UserDetails user) {
8         return userDetailsRepository.save(user);
9     }
10
11     public UserDetails updateUser(Long id, UserDetails user) {
12         Optional<UserDetails> existingUser =
13         userDetailsRepository.findById(id);
14         if (existingUser.isPresent()) {
15             /*
16             We are using the same "save" method to update,
17             internally it check, if its new entity or not, by looking
18             for ID field present in entity object. Since we are passing

```

```

18     ID in Request body, it will try to update instead of insert.
19     */
20     return userDetailsRepository.save(user);
21 }
22 return null;
23 }
24 }
25

```

1st INSERT OPERATION

POST localhost:8080/api/user

Body (JSON):

```

{
  "name": "JohnXYZ",
  "phone": "1234567890",
  "userAddress": {
    "street": "123 Street",
    "city": "Bangalore",
    "state": "Karnataka",
    "country": "India",
    "pinCode": "10001"
  }
}

```

SQL statement: SELECT * FROM USER_DETAILS

ADDRESS_ID	ID	NAME	PHONE
1	1	JohnXYZ	1234567890

(1 row, 3 ms)

SQL statement: SELECT * FROM USER_ADDRESS

ID	CITY	COUNTRY	PIN_CODE	STATE	STREET
1	Bangalore	India	10001	Karnataka	123 Street

(1 row, 1 ms)

2nd UPDATE OPERATION:

(Changes made in both UserDetails and UserAddress, but only UserDetails changes reflected)

PUT localhost:8080/api/user/1

Body (JSON):

```

{
  "id": 1,
  "name": "JohnXYZ_updated",
  "phone": "1234567890",
  "userAddress": {
    "id": 1,
    "street": "123 Street",
    "city": "Bangalore",
    "state": "Karnataka",
    "country": "India",
    "pinCode": "10001"
  }
}

```

SQL statement: SELECT * FROM USER_DETAILS

ADDRESS_ID	ID	NAME	PHONE
1	1	JohnXYZ_updated	1234567890

(1 row, 1 ms)

SQL statement: SELECT * FROM USER_ADDRESS

ID	CITY	COUNTRY	PIN_CODE	STATE	STREET
1	Bangalore	India	10001	Karnataka	123 Street

(1 row, 0 ms)

Now If I want to both INSERT and then UPDATE association capability, means I need both PERSIST + MERGE Cascade Type

```

1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)

```

```

7     private Long id;
8
9     private String name;
10    private String phone;
11
12    @OneToOne(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
13    @JoinColumn(name = "address_id", referencedColumnName = "id")
14    private UserAddress userAddress;
15
16    // Constructors
17    public UserDetails() {
18    }
19
20    // getters and setters
21 }
22

```

1st INSERT OPERATION , Similar like above.

POST localhost:8080/api/user

Body (JSON):

```

{
  "name": "JohnXYZ",
  "phone": "1234567890",
  "userAddress": {
    "street": "123 Street",
    "city": "Bangalore",
    "state": "Karnataka",
    "country": "India",
    "pinCode": "10001"
  }
}

```

Response Body (JSON):

```

{
  "id": 1,
  "name": "JohnXYZ",
  "phone": "1234567890",
  "userAddress": {
    "id": 1,
    "street": "123 Street",
    "city": "Bangalore",
    "state": "Karnataka",
    "country": "India",
    "pinCode": "10001"
  }
}

```

Run | Run Selected | Auto complete | Clear | SQL statement:

SELECT * FROM USER_DETAILS;

ADDRESS_ID	ID	NAME	PHONE
1	1	JohnXYZ	1234567890

(1 row, 3 ms)

Run | Run Selected | Auto complete | Clear | SQL statement:

SELECT * FROM USER_ADDRESS;

ID	CITY	COUNTRY	PIN_CODE	STATE	STREET
1	Bangalore	India	10001	Karnataka	123 Street

(1 row, 1 ms)

2nd UPDATE OPERATION

PUT localhost:8080/api/user/1

Body (JSON):

```

{
  "id": 1,
  "name": "JohnXYZ_updated",
  "phone": "1234567890",
  "userAddress": {
    "id": 1,
    "street": "123 Street",
    "city": "Bangalore",
    "state": "Karnataka",
    "country": "India",
    "pinCode": "10001"
  }
}

```

Response Body (JSON):

```

{
  "id": 1,
  "name": "JohnXYZ_updated",
  "phone": "1234567890",
  "userAddress": {
    "id": 1,
    "street": "123 Street",
    "city": "Bangalore",
    "state": "Karnataka",
    "country": "India",
    "pinCode": "10001"
  }
}

```

Run | Run Selected | Auto complete | Clear | SQL statement:

SELECT * FROM USER_DETAILS;

ADDRESS_ID	ID	NAME	PHONE
1	1	JohnXYZ_updated	1234567890

(1 row, 1 ms)

Run | Run Selected | Auto complete | Clear | SQL statement:

SELECT * FROM USER_ADDRESS;

ID	CITY	COUNTRY	PIN_CODE	STATE	STREET
1	Bangalore	India	10001	Karnataka	123 Street

(1 row, 2 ms)

CascadeType.REMOVE

- Deleting the User entity automatically delete its associated UserAddress entity data.

```

1 @RestController
2 @RequestMapping(value = "/api/")
3 public class UserController {

```



```

4
5     @Autowired
6     private UserDetailsService userDetailsService;
7
8     @PostMapping(path = "/user")
9     public UserDetails insertUser(@RequestBody UserDetails
10    userDetails) {
11         return userDetailsService.saveUser(userDetails);
12     }
13
14     @PutMapping(path = "/user/{id}")
15     public UserDetails updateUser(@PathVariable Long id,
16                                  @RequestBody UserDetails
17    userDetails) {
18         return userDetailsService.updateUser(id, userDetails);
19     }
20
21     @DeleteMapping("/{id}")
22     public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
23         userDetailsService.deleteUser(id);
24         return ResponseEntity.noContent().build();
25     }
26 }

```

```

1 @Service
2 public class UserDetailsService {
3
4     @Autowired
5     UserDetailsRepository userDetailsRepository;
6
7     public UserDetails saveUser(UserDetails user) {
8         return userDetailsRepository.save(user);
9     }
10
11     public UserDetails updateUser(Long id, UserDetails user) {
12         Optional<UserDetails> existingUser =
13             userDetailsRepository.findById(id);
14
15         if (existingUser.isPresent()) {
16             return userDetailsRepository.save(user);
17         }
18         return null;
19     }
20
21     public void deleteUser(Long userId) {
22         userDetailsRepository.deleteByUserId(userId);
23     }
24 }
25

```

```

1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String name;
10    private String phone;
11
12    @OneToOne(cascade = {
13        CascadeType.PERSIST,
14        CascadeType.MERGE,
15        CascadeType.REMOVE
16    })
17    @JoinColumn(name = "address_id", referencedColumnName = "id")
18    private UserAddress userAddress;
19
20    // Constructors

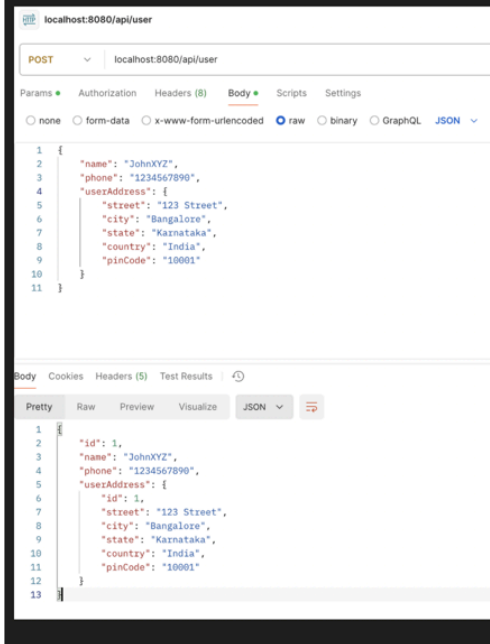
```

```

21 public UserDetails() {
22 }
23
24 // getters and setters
25 }
26

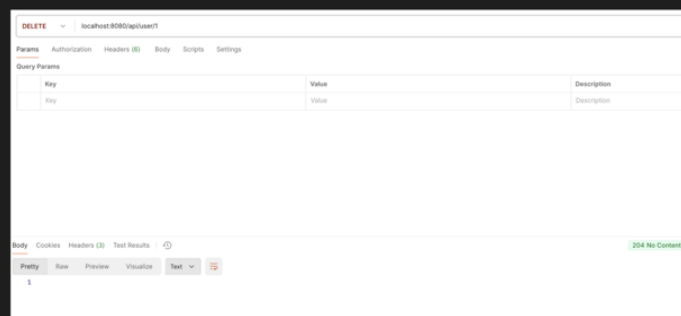
```

1st INSERT OPERATION , Similar like above.



Concept & Coding

2nd DELETE OPERATION



Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM USER_DETAILS

SELECT * FROM USER_DETAILS;
ADDRESS_ID ID NAME PHONE
(no rows, 1 ms)

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM USER_ADDRESS

SELECT * FROM USER_ADDRESS;
ID CITY COUNTRY PIN_CODE STATE STREET
(no rows, 1 ms)

CascadeType.REFRESH and CascadeType.DETACH

Both are Less frequently used.

CascadeType.REFRESH

```
1 @Service
2 public class UserDetailsService {
3
4     @Autowired
5     UserDetailsRepository userDetailsRepository;
6
7     public UserDetails saveUser(UserDetails user) {
8         return userDetailsRepository.save(user);
9     }
10
11     public UserDetails updateUser(Long id, UserDetails user) {
12         Optional<UserDetails> existingUser =
13             userDetailsRepository.findById(id);
14
15         if (existingUser.isPresent()) {
16             return userDetailsRepository.save(user);
17         }
18         return null;
19     }
20
21     public void deleteUser(Long userId) {
22         userDetailsRepository.deleteById(userId);
23     }
24 }
25
```

Concept && Coding

userDetailsRepository.save(user):-

Internally JPA code, has access to EntityManager object, which maintains persistenceContext and thus handles FIRST LEVEL CACHING

```
@Transactional
public <S extends T> S save(S entity) {
    Assert.notNull(entity, "Entity must not be null");
    if (this.entityInformation.isNew(entity)) {
        this.entityManager.persist(entity);
        return entity;
    } else {
        return this.entityManager.merge(entity);
    }
}
```

- Sometime, we want to BYPASS 'First Level Caching'
- So, EntityManager has one method called "refresh".
- What it does is, for that entity directly read the value from DB instead of First Level Cache.

```

EntityManager.java

/**
 * Refresh the state of the instance from the database,
 * overwriting changes made to the entity, if any.
 * @param entity entity instance
 * @throws IllegalArgumentException if the instance is not
 *         an entity or the entity is not managed
 * @throws TransactionRequiredException if there is no
 *         transaction when invoked on a container-managed
 *         entity manager of type <code>PersistenceContextType.TRANSACTION</code>
 * @throws EntityNotFoundException if the entity no longer
 *         exists in the database
 */
public void refresh(Object entity);

```

So, When we use the CascadeType.REFRESH, we tell JPA to not only read Parent Entity from DB but also its associated Child Entities too.

So in this case, whenever on USERDETAILS entity, entityManager will internally invoke 'refresh()' method', this Cascade type make sure that, USERADDRESS should also be read from DB not from First Level Cache.

```

1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String name;
10    private String phone;
11
12    @OneToOne(cascade = {CascadeType.REFRESH})
13    @JoinColumn(name = "address_id", referencedColumnName = "id")
14    private UserAddress userAddress;
15
16    // Constructors
17    public UserDetails() {
18    }
19
20    // getters and setters
21 }
22

```

CascadeType.DETACH

- Similarly like persist, remove, refresh method. EntityManager also has 'detach' method.
- Which purpose is to remove the given entity from the PERSISTENCE CONTEXT.
- Means JPA is not managing its lifecycle now.

```

/**
 * Remove the given entity from the persistence context, causing
 * a managed entity to become detached. Unflushed changes made
 * to the entity if any (including removal of the entity),
 * will not be synchronized to the database. Entities which
 * previously referenced the detached entity will continue to
 * reference it.
 * @param entity entity instance
 * @throws IllegalArgumentException if the instance is not an
 *         entity
 * @since 2.0
 */
public void detach(Object entity);

```

So, When we use the CascadeType.DETACH, we tell JPA to not only detach Parent Entity from persistence context but also its associated Child Entities too.

So in this case, whenever on USERDETAILS entity, entityManager will internally invoke 'detach() method', this Cascade type make sure that, USERADDRESS should also be detached/removed from persistence context.

```
1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String name;
10    private String phone;
11
12    @OneToOne(cascade = {CascadeType.DETACH})
13    @JoinColumn(name = "address_id", referencedColumnName = "id")
14    private UserAddress userAddress;
15
16    // Constructors
17    public UserDetails() {
18    }
19
20    // getters and setters
21 }
22
```

CascadeType.ALL

Concept && Coding

- Means we want all the different Cascade types capabilities like PERSIST, MERGE, REMOVE, REFRESH & DETACH.

Okay, we saw INSERT, UPDATE, REMOVE operation, but what about GET?

Does child entities always get loaded with Parent Entity?

It has 2 parts:-

1. Eager Loading
2. Lazy Loading

1. Eager Loading

- It means, associated entity is loaded immediately along with the parent entity.
- Default for @OneToOne and @ManyToOne

```

@Target({METHOD, FIELD})
@Retention(RUNTIME)

public @interface OneToOne {

    /**
     * (Optional) The entity class that is the target of the association.
     * Defaults to the type of the field or property that stores the association.
     */
    Class targetEntity() default void.class;

    /**
     * (Optional) The operations that must be cascaded to the target of the association.
     * By default no operations are cascaded.
     */
    CascadeType[] cascade() default {};

    /**
     * (Optional) Whether the association should be lazily
     * loaded or must be eagerly fetched. The EAGER
     * strategy is a requirement on the persistence provider runtime that
     * the associated entity must be eagerly fetched. The LAZY
     * strategy is a hint to the persistence provider runtime.
     */
    FetchType fetch() default FetchType.EAGER;
}

```

```

@Target({METHOD, FIELD})
@Retention(RUNTIME)

public @interface ManyToOne {

    /**
     * (Optional) The entity class that is the target of the association.
     * Defaults to the type of the field or property that stores the association.
     */
    Class targetEntity() default void.class;

    /**
     * (Optional) The operations that must be cascaded to the target of the association.
     * By default no operations are cascaded.
     */
    CascadeType[] cascade() default {};

    /**
     * (Optional) Whether the association should be lazily loaded or must be eagerly fetched. The EAGER
     * strategy is a requirement on the persistence provider runtime that the associated entity must be
     * eagerly fetched. The LAZY strategy is a hint to the persistence provider runtime.
     */
    FetchType fetch() default FetchType.EAGER;

    /**
     * (Optional) Whether the association is optional. If set to false then a non-null relationship must
     * always exist.
     */
    boolean optional() default true;
}

```

JPA, do an assumption that, since there is only 1 child entity present, so possibly it might be required while accessing parent entity

2. Lazy Loading

- It means, associated entity is NOT loaded immediately.
- Only loaded when explicitly accessed like when we call `userDetail.getUserAddress()`.
- Default for `@OneToMany`, `@ManyToMany`.

```

@Target({METHOD, FIELD})
@Retention(RUNTIME)

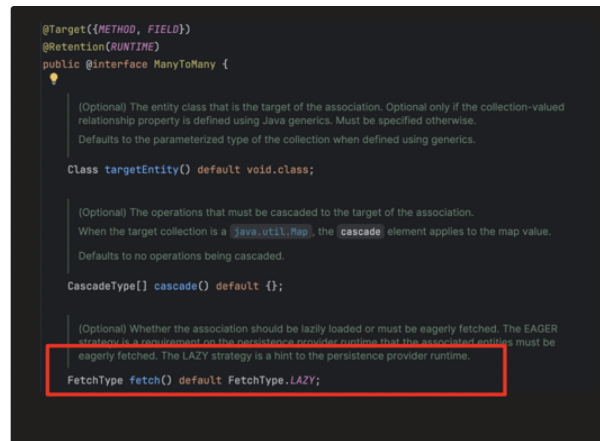
public @interface OneToMany {

    /**
     * (Optional) The entity class that is the target of the association. Optional only if the collection
     * property is defined using Java generics. Must be specified otherwise.
     * Defaults to the parameterized type of the collection when defined using generics.
     */
    Class targetEntity() default void.class;

    /**
     * (Optional) The operations that must be cascaded to the target of the association.
     * Defaults to no operations being cascaded.
     * When the target collection is a java.util.Map, the cascade element applies to the map value.
     */
    CascadeType[] cascade() default {};

    /**
     * (Optional) Whether the association should be lazily loaded or must be eagerly fetched. The EAGER
     * strategy is a requirement on the persistence provider runtime that the associated entities must be
     * eagerly fetched. The LAZY strategy is a hint to the persistence provider runtime.
     */
    FetchType fetch() default FetchType.LAZY;
}

```



But here, there can be many child entities present for a parent entity, so returning all the rows of the child entity might impact performance, so by-default it uses LAZY technique.

We can also control this default behavior:

```
1 @RestController
2 @RequestMapping(value = "/api/")
3 public class UserController {
4
5     @Autowired
6     private UserDetailsService userDetailsService;
7
8     @PostMapping(path = "/user")
9     public UserDetails insertUser(@RequestBody UserDetails
10     userDetails) {
11         return userDetailsService.saveUser(userDetails);
12     }
13
14     @GetMapping("/user/{id}")
15     public UserDetails fetchUser(@PathVariable Long id) {
16         return userDetailsService.findById(id);
17     }
18 }
```

```
1 @Service
2 public class UserDetailsService {
3
4     @Autowired
5     UserDetailsRepository userDetailsRepository;
6
7     public UserDetails saveUser(UserDetails user) {
8         return userDetailsRepository.save(user);
9     }
10
11     public UserDetails findById(Long primaryKey) {
12         return userDetailsRepository.findById(primaryKey).get();
13     }
14 }
15 }
```

```
1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String name;
```

```

10     private String phone;
11
12     @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
13     @JoinColumn(name = "address_id", referencedColumnName = "id")
14     private UserAddress userAddress;
15
16     // Constructors
17     public UserDetails() {
18     }
19
20     // getters and setters
21 }
22

```

Lets try testing this code:

1st Insert Operation: (Success)

POST localhost:8080/api/user

Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```

1 {
2   "name": "JohnXYZ",
3   "phone": "1234567890",
4   "userAddress": {
5     "street": "123 Street",
6     "city": "Bangalore",
7     "state": "Karnataka",
8     "country": "India",
9     "pinCode": "100001"
10  }
11 }

```

Body Cookies Headers (5) Test Results 200 OK

Pretty Raw Preview Visualize JSON

```

1 {
2   "id": 1,
3   "name": "JohnXYZ",
4   "phone": "1234567890",
5   "userAddress": {
6     "id": 1,
7     "street": "123 Street",
8     "city": "Bangalore",
9     "state": "Karnataka",
10    "country": "India",
11    "pinCode": "100001"
12  }
13 }

```

2nd Get Operation: (Failure)

GET localhost:8080/api/user/1

Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (4) Test Results 500 Internal Server Error

Pretty Raw Preview Visualize JSON

```

1 {
2   "timestamp": "2024-12-31T08:44:42.478+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/api/user/1"
6 }

```


GET operation failed because of loading UserAddress LAZILY, Since during JSON creation at the time of response, UserAddress data is missing thus library like JAKSON don't know how to serialize it and thus it failed while constructing the response.

```
com.fasterxml.jackson.databind.exc.InvalidDefinitionException: Could not find serializer for class org.hibernate.proxy.pojo.bytebuddy.ByteBuddyInterceptor
    at com.fasterxml.jackson.databind.exc.InvalidDefinitionException.from(InvalidDefinitionException.java:77) ~[jackson-databind-2.17.2.jar:2.17.2]
    at com.fasterxml.jackson.databind.SerializerProvider.reportBadDefinition(SerializerProvider.java:1330) ~[jackson-databind-2.17.2.jar:2.17.2]
    at com.fasterxml.jackson.databind.DatabindContext.reportBadDefinition(DatabindContext.java:414) ~[jackson-databind-2.17.2.jar:2.17.2]
```

Hey, then how come INSERT operation, we don't see this serialization issue?

Its because during INSERT, we are inserting the data in DB and also its inserted into persistence Context (first level cache), so UserAddress data is already present in-memory. So when response is returned as part of same INSERT operation, it do not make any DB call, just fetched from the persistence context.

So, for GET operation, how to solve it:

1. Use JSonIgnore

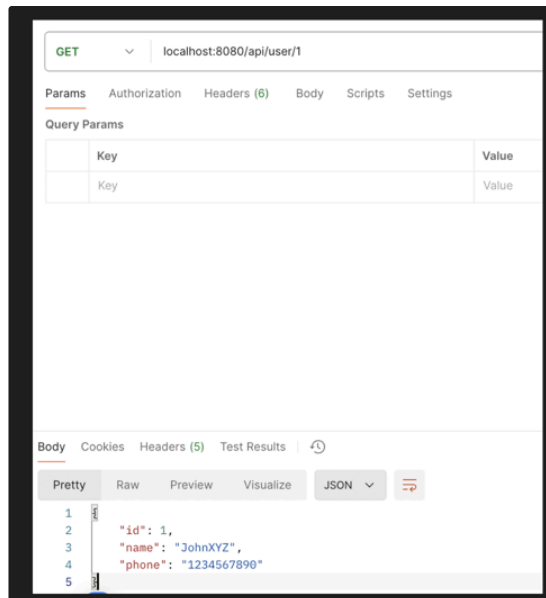
2. Use DTO (Data Transfer Object)

1. Using JSonIgnore

- This will remove the UserAddress field totally for both Lazy and Eager loading.

```
1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String name;
10    private String phone;
11
12    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
13    @JoinColumn(name = "address_id", referencedColumnName = "id")
14    @JsonIgnore
15    private UserAddress userAddress;
16
17    // Constructors
18    public UserDetails() {
19    }
20
21    // getters and setters
22 }
23
```

Concept && Coding



2. Using DTO(Data Transfer Object)

- Much clean and recommended approach.
- Instead of sending Entity directly, first response will be mapped to our DTO object.

```
1 @RestController
2 @RequestMapping(value = "/api/")
3 public class UserController {
4
5     @Autowired
6     UserDetailsService userDetailsService;
7
8     @PostMapping(path = "/user")
9     public UserDetails insertUser(@RequestBody UserDetails
10     userDetails) {
11         return userDetailsService.saveUser(userDetails);
12     }
13
14     @GetMapping("/user/{id}")
15     public UserDetailsDTO fetchUser(@PathVariable Long id) {
16         return userDetailsService.findById(id).toDTO();
17     }
18 }
```

```
1 @Service
2 public class UserDetailsService {
3
4     @Autowired
5     UserDetailsRepository userDetailsRepository;
6
7     public UserDetails saveUser(UserDetails user) {
8         return userDetailsRepository.save(user);
9     }
10
11     public UserDetails findById(Long primaryKey) {
12         return userDetailsRepository.findById(primaryKey).get();
13     }
14 }
15
```

```
1 public class UserDetailsDTO {
2
3     private Long id;
```

```

4     private String name;
5     private String phone;
6     private String address;
7
8     // Constructor to populate from UserDetails entity
9     public UserDetailsDTO(UserDetails userDetails) {
10         this.id = userDetails.getId();
11         this.name = userDetails.getName();
12         this.phone = userDetails.getPhone();
13         System.out.println("going to query user address here now");
14         this.address = userDetails.getUserAddress() != null ?
15             userDetails.getUserAddress().getStreet() : null;
16     }
17
18     // getters and setters
19 }
20

```

```

1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8     private String name;
9     private String phone;
10
11     @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
12     @JoinColumn(name = "address_id", referencedColumnName = "id")
13     private UserAddress userAddress;
14
15     // Constructors
16     public UserDetails() {
17     }
18
19     public UserDetailsDTO toDTO() {
20         return new UserDetailsDTO(this);
21     }
22
23     // getters and setters
24 }
25

```

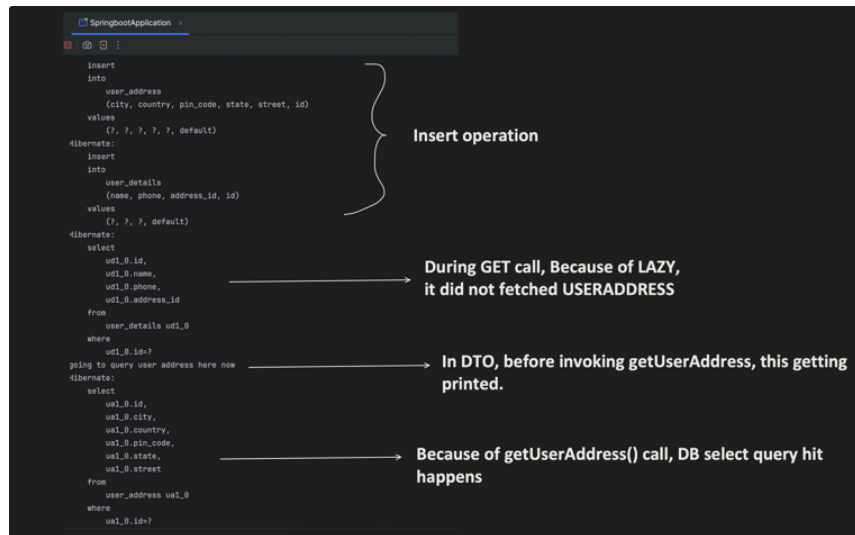
Concept & Coding

The screenshot shows a REST client interface with the following details:

- URL:** localhost:8080/api/user/1
- Method:** GET
- Query Params:** A table with two columns: Key and Value. The first row has 'Key' and 'Value' as placeholders.
- Body:** The response is displayed in JSON format, showing the following data:


```

      {
        "id": 1,
        "name": "JohnXYZ",
        "phone": "1234567890",
        "address": "123 Street"
      }
      
```



@OneToOne bidirectional

- Both entities hold reference to each other, means:
 - UserDetails has a reference to UserAddress.
 - UserAddress also has a reference back to UserDetails (only in Object, not in DB table)

Owner side

- Holds the Foreign Key relationship in a table.

Concept && Coding

```

1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8     private String name;
9     private String phone;
10
11     @OneToOne(cascade = CascadeType.ALL)
12     @JoinColumn(name = "address_id", referencedColumnName = "id")
13     private UserAddress userAddress;
14
15     // Constructors
16     public UserDetails() {
17     }
18
19     // getters and setters
20 }
21
  
```

Inverse side

- No Foreign key is created in table.
- Only holds **Object** reference of owing entity.

```

1 @Entity
2 @Table(name = "user_address")
3 public class UserAddress {
4
  
```

```

5      @Id
6      @GeneratedValue(strategy = GenerationType.IDENTITY)
7      private Long id;
8
9      private String street;
10     private String city;
11     private String state;
12     private String country;
13     private String pinCode;
14
15     @OneToOne(mappedBy = "userAddress", fetch = FetchType.EAGER)
16     private UserDetails userDetails;
17
18     // getters and setters
19 }
20

```

Table looks exactly same as OneToOne Unidirectional only



Concept & Coding

But we now have capability to go backward from UserAddress to UserDetails.

Created a controller and service class to query UserAddress entity

```

1  @RestController
2  @RequestMapping(value = "/api/")
3  public class UserAddressController {
4
5      @Autowired
6      UserAddressService userDetailsService;
7
8      @GetMapping("/user-address/{id}")
9      public UserAddress fetchUser(@PathVariable Long id) {
10         return userDetailsService.findById(id);
11     }
12 }
13

```

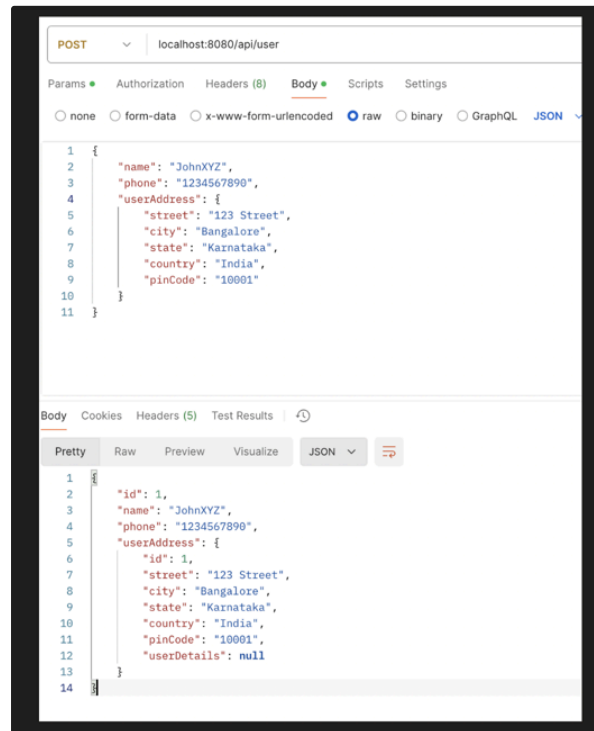
```

1  @Service
2  public class UserAddressService {
3
4      @Autowired
5      UserAddressRepository userAddressRepository;
6
7      public UserAddress findById(Long primaryKey) {
8         return userAddressRepository.findById(primaryKey).get();
9     }
10 }
11

```

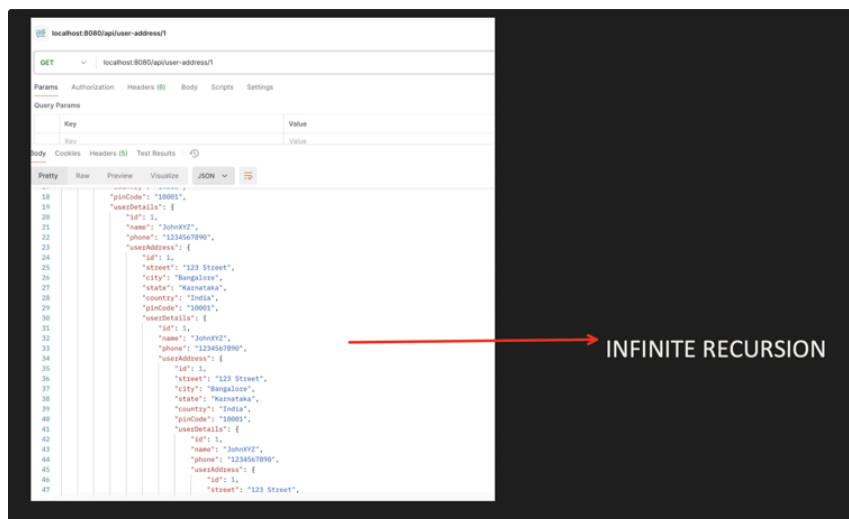
Let's observe the behavior now,

1st: Insert Operation like before, using UserDetails controller API:



```
2024-12-31T16:45:04.749+05:30 INFO 39163 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
Hibernate:
insert
into
  user_address
(city, country, pin_code, state, street, id)
values
  (?, ?, ?, ?, default)
Hibernate:
insert
into
  user_details
(name, phone, address_id, id)
values
  (?, ?, ?, default)
```

2nd: Get call of UserAddress Controller API:





Why because :

During response construction:

1. Jackson starts serializing UserAddress after UserAddress is serialized.
2. It encounters UserDetails within UserAddress and starts serializing it. After UserDetails is serialized.
3. Inside UserDetails , it encounters UserAddress again and serialization keeps going on in loop.

Infinite Recursion issue in bidirectional mapping can be solved via:

Concept && Coding

@JsonManagedReference and @JsonBackReference

@JsonManagedReference:

- Should be Used only in Owning entity.
- Tells explicitly Jackson to go ahead and serialize the child entity.

@JsonBackReference:

- Should only be Used with Inverse/Child entity.
- Tells explicitly Jackson to not serialize the parent entity.

```
1 @Table(name = "user_details")
2 @Entity
3 public class UserDetails {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String name;
10    private String phone;
11
12    @OneToOne(cascade = CascadeType.ALL)
13    @JoinColumn(name = "address_id", referencedColumnName = "id")
14    @JsonManagedReference
15    private UserAddress userAddress;
16
17    // Constructors
```

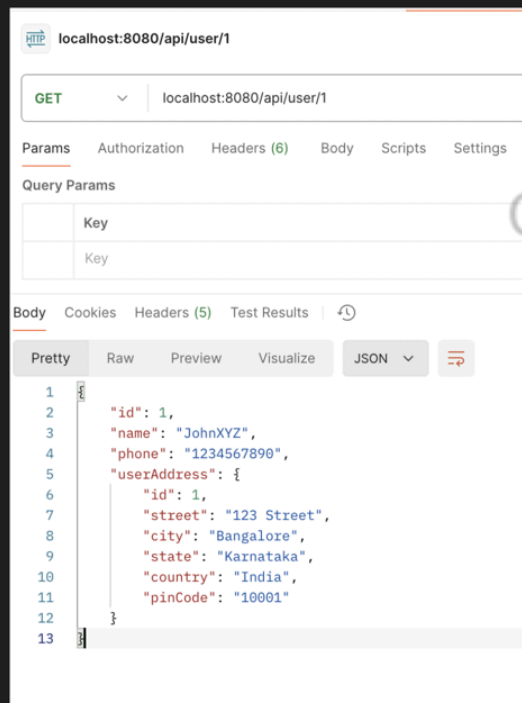
18

```

1 @Entity
2 @Table(name = "user_address")
3 public class UserAddress {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String street;
10    private String city;
11    private String state;
12    private String country;
13    private String pinCode;
14
15    @OneToOne(mappedBy = "userAddress", fetch = FetchType.EAGER)
16    @JsonBackReference
17    private UserDetails userDetails;
18
19    // getters and setters
20 }
21

```

GET API call on Parent Entity "UserDetail"



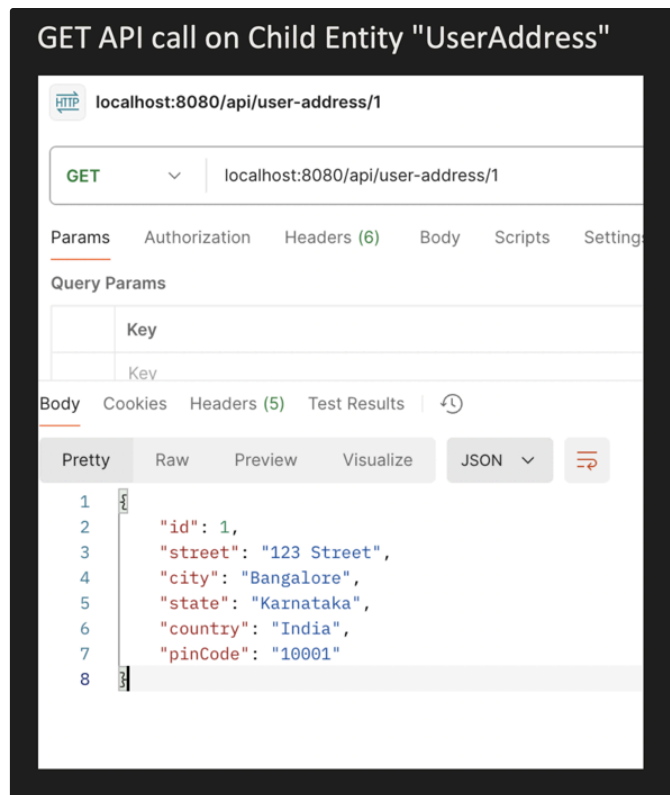
The screenshot shows a REST client interface with the following details:

- URL:** localhost:8080/api/user/1
- Method:** GET
- Params:** None
- Body:** Pretty view of the JSON response.


```

1 {
2   "id": 1,
3   "name": "JohnXYZ",
4   "phone": "1234567890",
5   "userAddress": {
6     "id": 1,
7     "street": "123 Street",
8     "city": "Bangalore",
9     "state": "Karnataka",
10    "country": "India",
11    "pinCode": "10001"
12  }
13 }

```

Is there a way that, I can load the associated entity from both side, but still avoid infinite recursion:

Concept & Coding

@JsonIdentityInfo

- During serialization , Jackson gives the unique ID to the entity (based on property field).
- Though which Jackson can know, if that particular id entity is already serialized before, then it skip the serialization.

```

1  @Table(name = "user_details")
2  @Entity
3  @JsonIdentityInfo(
4      generator = ObjectIdGenerators.PropertyGenerator.class,
5      property = "id"
6  )
7  public class UserDetails {
8
9      @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     /*
12     Any unique field from entity we can give, generally we give PK
13     */
14     private Long id;
15     private String name;
16     private String phone;
17
18     @OneToOne(cascade = CascadeType.ALL)
19     @JoinColumn(name = "address_id", referencedColumnName = "id")
20     private UserAddress userAddress;
21
22     // Constructors
23     public UserDetails() {
24     }
25
26     // getters and setters
27 }
28

```

```

1 @Entity
2 @Table(name = "user_address")
3 @JsonIdentityInfo(
4     generator = ObjectIdGenerators.PropertyGenerator.class,
5     property = "id"
6 )
7 public class UserAddress {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12
13    private String street;
14    private String city;
15    private String state;
16    private String country;
17    private String pinCode;
18
19    @OneToOne(mappedBy = "userAddress", fetch = FetchType.EAGER)
20    private UserDetails userDetails;
21
22    // getters and setters
23 }
24

```

GET API call on Parent Entity "UserDetail"

HTTP localhost:8080/api/user/1

GET localhost:8080/api/user/1

Params Authorization Headers (6) Body Scripts Settings

Query Params

Key
Key

Body Cookies Headers (5) Test Results

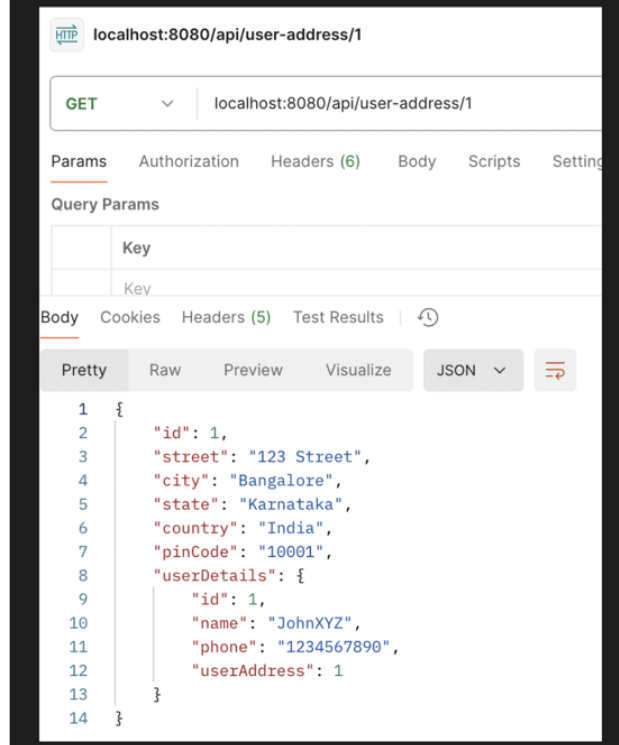
Pretty Raw Preview Visualize JSON

```

1 {
2     "id": 1,
3     "name": "JohnXYZ",
4     "phone": "1234567890",
5     "userAddress": {
6         "id": 1,
7         "street": "123 Street",
8         "city": "Bangalore",
9         "state": "Karnataka",
10        "country": "India",
11        "pinCode": "10001",
12        "userDetails": 1
13    }
14 }

```

GET API call on Child Entity "UserAddress"



localhost:8080/api/user-address/1

GET localhost:8080/api/user-address/1

Params Authorization Headers (6) Body Scripts Settings

Query Params

Key
Key

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 1,
3   "street": "123 Street",
4   "city": "Bangalore",
5   "state": "Karnataka",
6   "country": "India",
7   "pinCode": "10001",
8   "userDetails": {
9     "id": 1,
10    "name": "JohnXYZ",
11    "phone": "1234567890",
12    "userAddress": 1
13  }
14 }
```

Concept && Coding